

Reconsider Redeclaring static constexpr Data Members

Document #: P2984R0
Date: 2023-09-27
Project: Programming Language C++
Audience: Library Evolution
Reply-to: Alisdair Meredith
<ameredith1@bloomberg.net>

Contents

- 1 Abstract
- 2 Revision history
 - R0 October 2023 (pre-Kona mailing)
- 3 Introduction
- 4 History
 - 4.1 Introducing constexpr in C++11
 - 4.2 Introducing inline variables in C++17
 - 4.3 2023 Varna meeting: initial EWG review
- 5 Analysis
 - 5.1 What the C++ Standard says
 - 5.2 Workarounds
 - 5.3 Current practice
- 6 Proposal
 - 6.1 Status quo
 - 6.2 Undeprecate the redundant redeclarations for C++26
 - 6.3 Remove the deprecated redundant redeclaration from C++26
- 7 Acknowledgements
- 8 References

§ 1 Abstract

With the introduction of inline variables in C++17, `static constexpr` data members are defined by their in-class declarations and the legacy out-of-class definitions became redundant redeclarations and were deprecated. This paper examines the feasibility of removing support for the deprecated redeclarations, and whether undeprecation would be the better forward-looking policy.

§ 2 Revision history

§ R0 October 2023 (pre-Kona mailing)

— Initial draft of this paper

§ 3 Introduction

At the start of the C++23 cycle, [P2139R2] tried to review each deprecated feature of C++, to see which we would benefit from actively removing, and which might now be better undeprecated. Consolidating all this analysis into one place was intended to ease the (L)EWG review process, but in return gave the author so much feedback that the next revision of that paper was not completed.

For the C++26 cycle there will be a concise paper tracking the overall review process, [P2863R2], but all changes to the standard will be pursued through specific papers, decoupling progress from the larger paper so that delays on a single feature do not hold up progress on all.

This paper takes up the deprecated redeclaration of `constexpr` data members, D.6 [depr.static constexpr].

§ 4 History

§ 4.1 Introducing `constexpr` in C++11

The first C++ Standard to support `static constexpr` data members was C++11, with the introduction of the `constexpr` keyword.

§ 4.2 Introducing inline variables in C++17

Definitions for `static constexpr` data members outside the enclosing class definition became redundant redeclarations with the application of [P0386R2] for C++17, inline variables. Such redeclarations were consequently deprecated, although considered harmless. There has been no further progress on this topic in the last seven years.

§ 4.3 2023 Varna meeting: initial EWG review

At the Varna meeting in 2023, the deprecation of D.6 [\[depr.static.constexpr\]](#) was presented in the context of maintaining status quo, as presented in paper P2863R0.

While there was unanimous agreement that removal from C++26 seemed a bad idea, there was interest in soliciting a paper to propose undeprecation.

Poll: EWG is interested in un-deprecating defining inline constexpr class variables (P2863R0 section 6.6).

SF	F	N	A	SA
2	12	7	3	0

Result: Consensus to request a paper, which you are now reading.

§ 5 Analysis

§ 5.1 What the C++ Standard says

According to 9.2.6 [\[dcl.constexpr\]](#), static constexpr data members of a class are implicitly defined as inline variables:

- 3 A function or static data member declared with the constexpr or constexpr specifier is implicitly an inline function or variable.

According to 6.2 [\[basic.def\]](#), inline static data member declarations are also definitions (by inference of not being excluded), and the following bullets clarify that further declarations outside the class are just that, declarations and not definitions.

- 2 Each entity declared by a declaration is also defined by that declaration unless:
 - (2.3) — it declares a non-inline static data member in a class definition (11.4 [\[class.mem\]](#), 11.4.9 [\[class.static\]](#)),
 - (2.4) — it declares a static data member outside a class definition and the variable was defined within the class with the constexpr specifier (11.4.9.3 [\[class.static.data\]](#)) (this usage is deprecated; see D.6 [\[depr.static.constexpr\]](#)),

§ 5.2 Workarounds

When considering the removal of redundant definitions, it seems simple enough to support a code base that is common with C++11 and the feature removal (or warned deprecation) with a simple feature check:

C++11 and C++14	Portable with no deprecation
<pre>using Type = int;</pre>	<pre>using Type = int;</pre>

C++11 and C++14	Portable with no deprecation
<pre>struct Class { Type Member = {}; }; constexpr Type Class::Member;</pre>	<pre>struct Class { Type Member = {}; }; #ifdef __cpp_inline_variables constexpr Type Class::Member; #endif</pre>

Note that the Portable code works both before and after C++17, and whether or not the redundant redeclaration is deprecated (removing a warning) or ill-formed if support for redeclarations were removed from a future standard. Once a codebase establishes that its minimum supported dialect of C++ is C++17 or later the conditionally translated code can be removed completely.

§ 5.3 Current practice

As of writing this paper in September 2023, none of the 4 major compiler front ends report a deprecated-use warning on the example in the standard, including the latest trunk build for open source compilers.

Example code from D.6 [\[depr.static constexpr\]](#):

```
struct A {
    static constexpr int n = 5; // definition (declaration in C++ 2014)
};

constexpr int A::n; // redundant declaration (definition in C++ 2014)
```

All the latest compiler available through Godbolt compiler explorer were tested with `-Wall -Wextra` except MSVC which was tested with `\w4`. Language dialect tested was C++20, as that is a common base for all compilers, and this feature has been deprecated since the earlier C++17.

Compiler	Warn since
Clang	no warnings
GCC	no warnings
MSVC	no warnings
nvc++/EDG	no warnings

§ 6 Proposal

We might consider several directions to make progress for C++26.

§ 6.1 Status quo

Status quo prevails if we do not make a persuasive enough argument to make a change. However, in this case there is an argument to be made explicitly in favor of the status quo.

To maintain maintain a simpler language in the long term, it would be nice to remove a corner case that permits redundant redeclarations where redeclarations are normally ill-formed. Removing corner cases that serve no benefit to the modern language avoids the accumulation of “cruft” that degrades the user experience in long-term ongoing project.

The workaround to make code portable across all C++ dialects is simple and amenable to tooling that can parse C++, such as through a fix-it hint from a compiler front end. If we retain the ambition to one day remove this deprecated feature, we should encourage compiler vendors to start diagnosing code that relies on it today. Such diagnostics have been relevant since 2017, and should be deployed without waiting for the release of the C++26 Standard.

If nothing else, retaining the deprecated status indicates that the feature is historical baggage and not an essential part of the language.

§ 6.2 Undeprecate the redundant redeclarations for C++26

If we believe that this deprecated feature can never be removed, then it would be an active disservice to our users for compilers and other tools to start warning on deprecated usage. In such case, we should actively consider undeprecating redundant redeclaration of static constexpr data members.

§ 6.2.1 Proposed wording

Make the following changes to the C++ Working Draft, undeprecating redundant redeclaration of constexpr data members outside their class. All wording is relative to [N4958], the latest draft at the time of writing.

§ 6.2.1.1 Remove deprecation notices from the core clauses

§ 6.2 [basic.def] Declarations and definitions

2 Each entity declared by a declaration is also defined by that declaration unless:

- (2.4) — it declares a static data member outside a class definition and the variable was defined within the class with the constexpr specifier (11.4.9.3 [class.static.data]) (~~this usage is deprecated; see D.6 [depr.static constexpr]~~),

§ 11.4.9.3 [class.static.data] Static data members

- 4 If a non-volatile non-inline const static data member is of integral or enumeration type, its declaration in the class definition can specify a *brace-or-equal-initializer* in which every *initializer-clause* that is an *assignment-expression* is a constant expression (7.7

[[expr.const](#)]). The member shall still be defined in a namespace scope if it is odr-used (6.3 [[basic.def.odr](#)]) in the program and the namespace scope definition shall not contain an *initializer*. The declaration of an inline static data member (which is a definition) may specify a *brace-or-equal-initializer*. If the member is declared with the `constexpr` specifier, it may be redeclared in namespace scope with no initializer (~~this usage is deprecated; see D.6 [[depr.static constexpr](#)]~~). Declarations of other static data members shall not specify a *brace-or-equal-initializer*.

§ 6.2.1.2 Review Annex C

No changes are needed for Annex C, as restoring deprecated functionality does not risk any breakage.

§ 6.2.1.3 Strike wording from Annex D

§ D.6 [[depr.static constexpr](#)] Redeclaration of static constexpr data members

- 1 For compatibility with prior revisions of C++, a `constexpr` static data member may be redundantly redeclared outside the class with no initializer (6.2 [[basic.def](#)], 11.4.9.3 [[class.static.data](#)]). This usage is deprecated.

[*Example 1:*

```
struct A {  
    static constexpr int n = 5;    // definition (declaration in C++ 2014)  
};  
constexpr int A::n;                // redundant declaration (definition in  
    C++ 2014)
```

—*end example*]

§ 6.2.1.4 Update cross-reference for stable labels for C++23

§ Cross-references from ISO C++ 2023

All clause and subclause labels from ISO C++ 2023 (ISO/IEC 14882:2023, *Programming Languages — C++*) are present in this document, with the exceptions described below.

container.gen.reqmts see
 container.requirements.general

depr.res.on.required *removed*
[depr.static constexpr](#) *removed*

§ 6.3 Remove the deprecated redundant redeclaration from C++26

Given the lack of deprecation warnings issued at the time of writing, 6 years after the redundant redeclarations were formally deprecated in a published standard, it seems unreasonable to recommend removal at this time. Since the deprecated syntax was actually required by C++11 and C++14 (prior to C++11 there was no `constexpr` keyword) it is likely that a lot of current code would break upon removing this feature without a transitional period where compilers do issue warnings about the current deprecation.

That said, the workaround to make code portable across all C++ dialects is simple and amenable to tooling that can parse C++, such as through a fix-it hint from a compiler front end.

§ 6.3.1 Proposed wording

Make the following changes to the C++ Working Draft, making redundant redeclaration of `constexpr` data members outside their class ill-formed. All wording is relative to [N4958], the latest draft at the time of writing.

§ 6.2 [basic.def] Declarations and definitions

- 2 Each entity declared by a declaration is also defined by that declaration unless:

(2.4) ~~— it declares a static data member outside a class definition and the variable was defined within the class with the `constexpr` specifier (11.4.9.3 [class.static.data]) (this usage is deprecated; see D.6 [depr.static constexpr]),~~

§ 11.4.9.3 [class.static.data] Static data members

- 4 If a non-volatile non-inline `const` static data member is of integral or enumeration type, its declaration in the class definition can specify a *brace-or-equal-initializer* in which every *initializer-clause* that is an *assignment-expression* is a constant expression (7.7 [expr.const]). The member shall still be defined in a namespace scope if it is odr-used (6.3 [basic.def.odr]) in the program and the namespace scope definition shall not contain an *initializer*. The declaration of an inline static data member (which is a definition) may specify a *brace-or-equal-initializer*. ~~If the member is declared with the `constexpr` specifier, it may be redeclared in namespace scope with no initializer (this usage is deprecated; see D.6 [depr.static constexpr]).~~ Declarations of other static data members shall not specify a *brace-or-equal-initializer*.

§ 6.3.1.1 Update Annex C

TBD

[Example 1:

```
struct A {  
    static constexpr int n = 5;    // inline variable definition  
};  
constexpr int A::n;                // ill-formed redeclaration in C++ 2026
```

—end example]

§ 6.3.1.2 Strike wording from Annex D

§ D.6 [depr.static.constexpr] Redclaration of static constexpr data members

- 1 For compatibility with prior revisions of C++, a constexpr static data member may be redundantly redeclared outside the class with no initializer (6.2 [basic.def], 11.4.9.3 [class.static.data]). This usage is deprecated.

[Example 1:

```
struct A {  
    static constexpr int n = 5;    // definition (declaration in C++ 2014)  
};  
constexpr int A::n;                // redundant declaration (definition in  
    C++ 2014)
```

—end example]

§ 6.3.1.3 Update cross-reference for stable labels for C++23

§ Cross-references from ISO C++ 2023

All clause and subclause labels from ISO C++ 2023 (ISO/IEC 14882:2023, *Programming Languages — C++*) are present in this document, with the exceptions described below.

container.gen.reqmts *see*
 container.requirements.general

depr.res.on.required *removed*
[depr.static.constexpr removed](#)

§ 7 Acknowledgements

Thanks to Michael Park for the pandoc-based framework used to transform this document's source from Markdown.

§ 8 References

- [N4958] Thomas Köppe. 2023-08-14. Working Draft, Programming Languages — C++. <https://wg21.link/n4958>
- [P0386R2] Hal Finkel, Richard Smith. 2016-06-24. Inline Variables. <https://wg21.link/p0386r2>
- [P2139R2] Alisdair Meredith. 2020-07-15. Reviewing Deprecated Facilities of C++20 for C++23. <https://wg21.link/p2139r2>

[P2863R2] Alisdair Meredith. 2023-10-15. Review Annex D for C++26.

<https://wg21.link/d2863r2>